

计算机系统导论（ICS）

Introduction to Computer System

Author: Simson

Date: September 2026

目录

1 基本介绍	1
1.1 ICS 的主要内容	1
1.2 CSAPP 的主要章节	1
2 信息的表示和处理	2
2.1 信息的二进制表示	2
2.1.1 信息量的单位	2
2.1.2 Boolean Algebra	2
2.1.3 数据的表示	2
2.2 浮点数	3
2.2.1 二进制分数 Fractional Binary Numbers	3
2.3 C 语言的运算	5
2.3.1 算术运算	5
2.3.2 逻辑运算	5
2.3.3 移位运算	5
2.3.4 无符号数运算	6
2.3.5 浮点数运算	6
2.4 信息的表示和存储	7

1 基本介绍

本课程非常著名，使用的教材是 CSAPP 《深入理解计算机系统》

1.1 ICS 的主要内容

- 机器语言和编译（编译优化生成机器语言）
- 程序性能评估和优化
- 存储结构组织和管理
- 网络技术和协议
- 并行计算相关知识

1.2 CSAPP 的主要章节

1. 数据类型及其表示：了解计算规律和数学算术规律的差别
2. 汇编程序和程序的机器级表示
 - 了解程序执行模型，查找相关的程序错误（底层汇编代码工具）
 - 程序性能调优
 - 软件开发和系统软件
 - 安全性分析（汇编代码静态分析）
3. 处理器体系结构
4. 存储器层次结构：程序和数据在栈上的分布
5. 异常控制流（定制 shell 程序实验）
 - 进程控制
 - 信号和信号处理
6. 虚拟内存：数据的布局和数据组织
7. 算法性能分析
 - 内存性能影响程序性能
 - 程序编写符合缓存大小和性能
8. 计算机网络和并发编程（代理服务器）

2 信息的表示和处理

2.1 信息的二进制表示

Reason : Binary presentation is the most practical system to use.

2.1.1 信息量的单位

bits 是最基本的单位。

Byte = 8 bits

2.1.2 Boolean Algebra

逻辑运算的代数表示, 与, 或, 非, 异或。

用 boolean 代数的表示方法, 可以将与或非异或运算对应到交并补差等集合运算

2.1.3 数据的表示

Unsigned 数据 直接转换二进制存储

$$B2U(X) = \sum_{i=0}^{w-1} 2^i$$

范围:

$$Umin = 0, \quad Umax = 2^w - 1$$

Signed 数据 负数原来使用源码表示, 正负 0 不好区分等问题最终弃用, 改为补码(Two's Complement)。最高位为符号位, 0 为正数, 1 为负数

$$B2T(X) = -2^{w-1} + \sum_{i=0}^{w-2} 2^i$$

负数的补码等于模减去该负数的绝对值; 对于某一确定的模, 某数减去小于模的另一个数, 总可以用另一个数字加上该数字的补码来确定

范围:

$$Tmin = -2^{w-1}, \quad Tmax = 2^{w-1} - 1$$

Example

10 → 01010

-10 → 10110

-1 → 11111111...

(全是 1)

在编程中，有不同计算机的 long 类型有关键字可以跨平台进行。

```
1 #include <limits.h>
2
3 LONG_MAX
4 LONG_MIN
```

在 C 语言中，常量如果不加 U，则默认为有符号数。加 U 则认为无符号数。

类型的转换相当于先转换为二进制表示，再对于同一个二进制串使用不同的方式进行解读。在进行数值比较的时候，如果有无符号数同时出现，自动转换为无符号数字。

存储大小如果变大，在高位复制填入符号位直到填满为止，操作称为扩充。存储大小如果缩小，直接按照所需的位数进行溢出截断，相当于对 2^n 进行取模运算

2.2 浮点数

浮点数是大模型数值表示的物理基础，大模型的很多工程技巧本质上都是与浮点数的局限性做折中权衡。训练和推理大量使用低精度浮点数，大模型训练几乎都使用混合参数。大模型部署的时候经常做量化，比如转换 FP16 为 INT8 或者 INT4，虽然目标格式是整数，但是整个过程依赖于对浮点数值分布的理解。

2.2.1 二进制分数 Fractional Binary Numbers

二进制分数数学表示 二进制分数的基本形式： 1011.11_2 ，二进制分数和二进制整数的表示遵从同样的规律：

$$x = \sum_{n=-j}^i 2^n$$

分数的乘除法和整数一样，都是靠小数点移位来实现的。需要注意一个数值的特殊表示，即 $0.111111111\ldots_2$ ，它总是小于 1，记为 $1 - \epsilon_0$ 。

二进制分数具有一定的范围局限性：只能精确表示小数部分可以被拆分为 0.5 的幂次之和的。其他的有理数会变成二进制循环小数。同时由于数据类型的大小限制，只能表示一定范围和一定精度的浮点数。

浮点数表示的 IEEE 标准 浮点数标准来源于 IEEE Standard754，在此之前有很多个人的独立表示。IEEE754 标准适合于所有主流 CPU，不同数据类型的转换过程中会存在很多错误（譬如 Ariana 的坠机来源于将一个 64 位浮点数转换为 16 位带符号整数时，产生了溢出异常）。下面对浮点数的表示技巧进行讲解。

浮点数表示方法来源于二进制科学记数法，以一个数字为例：

$$1.10110011001012_2 \times 2^13$$

根据二进制科学计数法表示，可以将二进制小数拆分为三个部分表示。

$$(-1)^s M 2^E$$

s 为符号表示， M 为有效位， E 为指数位置。编码的时候按照数据类型的具体大小分配到三部分进行编码。

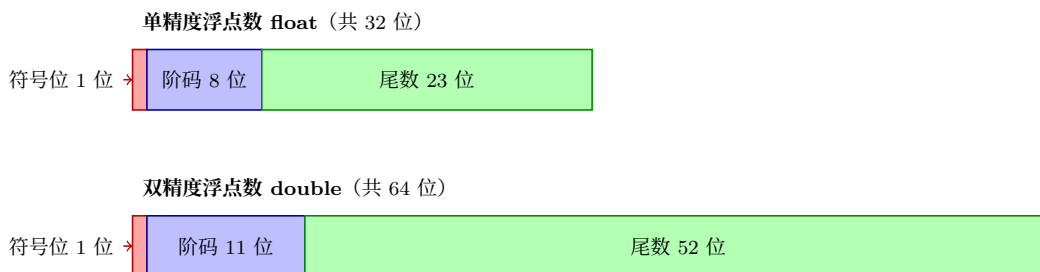


图 1: 单精度与双精度浮点数的存储结构（两行按同一比例绘制，条形长度正比于位数，故双精度条长为单精度的两倍）

三类位模式 位域划分定下来之后，三段具体取什么值，由阶码 exp 决定。所有位模式分为三类：

阶码 exp	尾数 $frac$	类别	数值
全 0	全 0	± 0	± 0
全 0	非全 0	非规格化数 (Denormalized)	$(-1)^s \times 0.f \times 2^{1-bias}$
非全 0 且非全 1	任意	规格化数 (Normalized)	$(-1)^s \times 1.f \times 2^{exp-bias}$
全 1	全 0	$\pm \infty$	$\pm \infty$
全 1	非全 0	NaN	非数 (Not a Number)

规格化 (Normalized) 数是最常见的一类，三个字段这样解释：

- $E = exp - bias$ ，其中偏置值 $bias = 2^{k-1} - 1$ ， k 为 exp 字段的位数。单精度 $k = 8$ ， $bias = 127$ ；双精度 $k = 11$ ， $bias = 1023$ 。
- $M = 1 + f$ ， f 是 $frac$ 字段的位模式所表示的小数部分。也就是说尾数最高位那个 1 是**隐含的**，不占存储空间。

偏置值取 $2^{k-1} - 1$ ，是为了让阶码的取值范围大致对称地跨在 0 两侧；同时这使得按无符号整数去比较 exp 字段，就等价于比较两个正浮点数的大小，硬件比较电路因此可以简化。

非规格化 (Denormalized) 数： exp 全 0 时，规定 $E = 1 - bias$ （而不是 $0 - bias$ ，这样才能与规格化数的边界平滑衔接），同时 $M = f$ ，**不再有隐含的最高位 1**。

- $frac$ 全 0：表示数值 0，但符号位仍然有效，因此有 $+0$ 与 -0 之分（见下）。
- $frac$ 非全 0：表示最接近 0 的那些数。单精度最小的非规格化数是 2^{-149} 。

特殊值： exp 全 1 时，若 $frac$ 全 0 则表示无穷 ($\pm \infty$)，若 $frac$ 非全 0 则表示 NaN (Not a Number, 非数)。

正零与负零 *exp* 全 0 且 *frac* 全 0 时表示数值 0，但符号位并不因此失效，于是同一个数值 0 对应两个不同的位模式：IEEE 754 规定 $+0 = -0$ ，C 语言中 $0.0 == -0.0$ 求值为真。所以**不能用 `==` 来区分正负零**，要判断符号得用 `signbit()`，或者去看 $1/x$ 的符号。**参与运算时二者不等价**。符号位在算术中照常起作用：

- 除法： $1/(+0) = +\infty$ ，而 $1/(-0) = -\infty$ 。
- 乘法：结果的符号由两个操作数的符号**异或**决定，所以 $(-3.0) \times (+0) = -0$ ，而 $(-0) \times (-0) = +0$ 。
- 加法：在就近舍入模式下 $(-0) + (+0) = +0$ 。
- 取平方根： $\sqrt{-0} = -0$ 。

保留 -0 目的是让符号信息在运算中连续传播，不至于中途丢失。在数值分析和复变函数里， -0 表示“从负方向趋近于 0”这一极限方向： $x \rightarrow 0^-$ 时 $1/x \rightarrow -\infty$ ， $\log(-0) = -\infty$ 。如果只保留 $+0$ ，左右极限就无法区分。整数采用补码，不存在负零。整数因此弃用了符号-幅值表示。

C 语言中注意区分两个写法： -0.0 是合法的浮点字面量，得到负零；而 -0 是整数常量，它的值就是 0，不存在负零。此外 `printf("%f", -0.0)` 会输出 -0.000000 ，负零的符号是可以被打印出来的。

2.3 C 语言的运算

C 语言中，将数据看作二进制向量进行处理。

- 算术运算
- 逻辑运算
- 移位运算

2.3.1 算术运算

和位相关，直接按位进行 Boolean 代数运算。

2.3.2 逻辑运算

主要是逻辑与，逻辑或

2.3.3 移位运算

左移：右侧补 0 右移：分算术右移和逻辑右移，逻辑右移直接向右移动高位补 0，算术右移右移补符号位

2.3.4 无符号数运算

无符号加法 首先是无符号数字的加法，首先是二进制加法，有可能发生溢出，相当于取模运算，最多取模一次

$$Uadd(u, v) = u + v \bmod 2^w$$

有符号数字的加法在 bit 位上有与无符号数完全相同的行为。如果计算结果超出了范围，可能发生正溢出，也可能发生负溢出。

$$TAdd_w(u, v) = \begin{cases} u + v + 2^w, & u + v < TMin_w \quad (\text{NegOver}) \\ u + v, & TMin_w \leq u + v \leq TMax_w \\ u + v - 2^w, & TMax_w < u + v \quad (\text{PosOver}) \end{cases}$$

无符号数乘法 乘法发生溢出的情况可能更多，可能需要算法包进行扩展，增大存储范围。

$$Umult(u, v) = u \cdot v \bmod 2^w$$

由于可能发生正负溢出，所以 TMult 有时候和 UMult 相比会出现不同的正负性。

乘法的具体操作是使用左右移动的方法，将乘数分解为二进制。左移 k 位，相当乘 2 的 k 次幂。

整数除法 除法相对于乘法，采用右移的方式来实现。除法涉及到向上取整的问题。

2.3.5 浮点数运算

基本的思路是，首先精确计算结果，然后根据格式做变换和解释。可能会根据计算情况进行溢出截断和舍入操作。

$$x +_f y = Round(x + y)$$

$$x \times_f y = Round(x \cdot y)$$

舍入规则 浮点数舍入规则为向偶数舍入。第一规则是向最近的数字靠拢，如果是中间数值距离是一样的，要向偶数舍入。其他的方法尤其是在求和的情况下有舍入的偏差，应当确认向最低的偶数位进行舍入，保证大规模运算的统计均值与真实数值一致。

计算机在执行舍入的时候，会设置在最低位进行按规则舍入，对于形如 BBGRXXX，G 为哨兵位即最低有效位；R 为 Round 位具体执行舍入。Sticky bit 在后面是要被扔掉的数字，用 Sticky bits 判断应该朝哪边进位。这一过程成为 Postnormalize，在浮点数解释完成之后处理。

乘法规则 浮点数乘法规则非常简单，符号位异或，指数位相加，小数 frac 位相乘。

如果乘法结果大于 2，则指数数位 +1，如果 E 超出了范围，报溢出错误。M 应当舍入到合适的精度。

加法规则 浮点数加法规则难于乘法，由于加法应当满足阶数对齐规则。不妨假设参与运算的浮点数满足 $E_1 > E_2$ 。通过移位运算（阶数小的数字向右移），将两个数的阶对齐。对齐之后应该移位就移位，该舍入就舍入，该报错就报错。

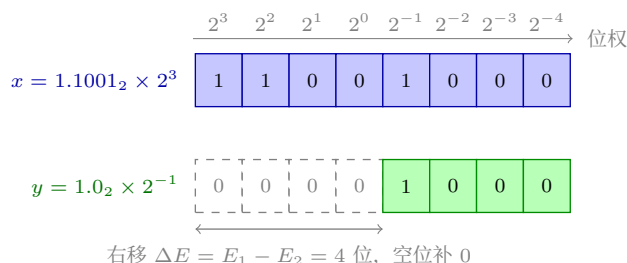


图 2: 浮点数加法的对阶过程（以 $12.5 + 0.5$ 为例）。两行条形图共用同一条位权轴： y 的阶小，尾数右移 $\Delta E = 4$ 位（虚线格为补入的 0），使两个数落在同一组位权上，再按列相加得 $1.1010_2 \times 2^3 = 13$

运算性质 浮点数加法和 Abel 群对比，对加法封闭，但是不支持结合律。除了 Nan 值和无穷值，所有元素都有加法逆元，除了特殊值满足不等式的加法性质。

浮点数和交换环相比，乘法满足交换性，由于溢出不满足结合率，同样不满足分配率，具有么元 1，不满足单调性，除了特殊值满足不等式的乘法性质。

类型转换 将浮点数转换为整数：向 0 舍入，如果是未定义怎么转换的数值默认转换为 Tmin。float 有自身的转换规则，所以转换过程中会有舍入损失。

2.4 信息的表示和存储

内存就是一个大型的数组，以 word 为单位存储。程序通过地址访问内存中的程序。内存中最基本的单位是字，一个地址占据的空间大小称为字长；这个参数决定了地址在内存中的分布。系统会按照字节进行编址和寻址。常见的系统是 32-bit 或者 64-bit，指的是一个地址空间对应的比特位。

一个多字节的数据，例如一个整数，占据 4 个字节。根据多字节数据在内存中排布的字节顺序，决定了两种存储模式。大端法指的是：最小的有效位置放在最大的地址，反之小端法最小的有效数字放在最小地址（地址的标示是整数），需要注意的是 Internet 的存储方式是使用大端法。

每个字符只占据一个字节，所以字符串的存储不受到大小端法的影响，可以用字符串起到标示和检查的效果。